
B28: SQL SuperHero Program : Session 45

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

11 May 2025 at 07:32

There is no thumb rule of performance tuning. It is very much possible that one query is performing well in one server and not performing good on another server. Tuning is all about how much time my business can wait. If My business can wait for 5 min and one query is taking 5 min, that is all okay, but if business can wait only 30 sec and my query is taking 35 sec to fetch data, so it means we need to tune the query. So tuning is not about how much time query taking, it is about how much my business can wait. [IRCTC Website Example VS ATM Transaction Example] [Black Friday website slow, but everyone just want transaction complete]

my sql is not running properly. It is taking time. What should you do?

Answer:

sir, if sql is not running properly, I have to ask 2 questions to the reporter:

1. since when you have this problem
it is starts today only, like till yesterday everything is fine and today the query is stuck for last 30 min
2. The query was written long back and everything was fine. as the time passes, query performance degrades. like 6 months before it is taking 5 sec, then 10 sec, last month taking 30 sec, last week 5 min, and today stuck since last 30 min

If suppose 2nd scenario, means query was previously working fine and as time passes it performance degrades.

solution: it is all because of data. As the data increases in the system, the performance is slow and query starts taking time. so we can do partitioning. The partition is a sure shot solution of performance. It can improve scalability and availability. query starts coming fast.

If the first scenario like yesterday's query is fine and today only its taking time. We have to do the below steps to understand the query performance:

1. check the explain plan of query, it gives us idea how the query performing / do all indexes are useful or not/ is there any full table scan or not
2. if there is any full table scan, we have to check why not following the index, why FTS (full table scan is showing)
3. we can use hint to follow index if required
4. If the index is not created/corrupted we can rebuild the index (drop and create index is better option)
5. we have to check the last analyzed date of table and indexes used by query (user_tables/user_indexes have last_analyzed column, which show the last_analyzed date. if this date is very old then)
6. ask dba to compute the statistics of the table and it will improve the performance. Even when we do bulk insert/update/.delete, it is advisable to analyze index and table to perform well. (**analyze table <tablename> compute statistics; analyze index <index name> compute statistics;**) -- book example. **COMPUTE STATISTICS instructs Oracle Database to compute exact statistics about the analyzed object and store them in the data dictionary. When you analyze a table, both table and column statistics are collected.**
7. Every Foreign Key must be indexed
8. we have to check query , is there any possibility to rewrite the query, like if in where clause like operator is used that can be replaced by instr because it is faster
9. we can replace IN by exists because exists clause is faster
10. we can see what other is taking time
11. explain plan is first way to start analysis as a developer to find out the root clause

FK Must be Indexed

For better performance of query, it is always advisable that every FK must be indexed.

As per oracle, We've been having some Oracle deadlock issues that have been hard to reproduce locally. while we've been pretty good creating integrity constraints in the database we have not been very good about making sure that every foreign key has a corresponding index. And that can lead to problems.

For example: So we had a situation where our documents table had a foreign key on the accounts table that was not indexed. So updating an account row lead to a whole table lock on documents (instead of just a row lock which would have happened if there

was an index) and that was very bad when we had two separate processes where one was doing a bunch of accounts stuff and the other was doing a lot of documents stuff. Deadlocks for everyone!

--> Script is attached along with this email, so check how many such FK exists in our database where we do not have an index.

=====

Performance Tuning is a practice to improve SQL queries in a way it runs as fast as possible and streamlines the database performance.

Basic operations to be performed are:

- DML
- DQL

While performing DML if a table is to be loaded with huge data, you can get good performance with BULK operation. BULK Collect and FORALL is introduced with oracle programming to enhance the performance of DML. operation
If using BULK insert – Use optimizer hint ‘APPEND’ and ‘NO LOGGING’, if redo buffer information is not required. This makes the insert operation faster.

- **Direct-load INSERT with logging:** This mode does full redo logging for instance and media recovery. Logging is the default mode.
- **Direct-load INSERT with no-logging:** In this mode, data is inserted without redo or undo logging.

- Redo and undo are two critical components that help manage data integrity and recovery.
- Redo refers to the mechanism that ensures all changes made to the database are recorded so that they can be reapplied in the event of a failure.
- Undo refers to the mechanism that allows Oracle to reverse changes made by uncommitted transactions.

Redo Log

The redo log buffer is a memory structure in Oracle that temporarily stores redo entries.

These entries record changes made to the database, such as inserts, updates, and deletes. It is located in the System Global Area (SGA).

The redo log buffer ensures that all changes made to the database can be recovered in the event of a system failure.

If a transaction is committed, the changes are recorded in the redo log buffer before they are written to the physical redo log files on disk.

Undo Buffer

The undo buffer is a memory structure that temporarily holds undo data, which is used to revert changes made by uncommitted transactions.

It ensures that the database can maintain a consistent state.

NOT VERY IMP. (JUST FOR NOTES)

Feature	Redo	Undo
Purpose	To reapply committed changes	To reverse uncommitted changes
Storage	Stored in redo log files	Stored in undo tablespaces
Operation	Used during recovery after a crash	Used for rollbacks during transactions
Example	Committed updates are logged	Changes rolled back to previous state

ALTER TABLE Tablename NOLOGGING;

use Direct Load Insert, add the APPEND hint to your INSERT statement. The combination of Direct-Path INSERT and NOLOGGING supports a very efficient load performance. But the price for it is that you are not able to recover lost data from the

archive redo log files.

```
INSERT /*+ APPEND*/  
INTO my_table  
SELECT * FROM my_other_table
```

A major benefit of direct-load INSERT is that you can load data without logging redo or undo entries, which improves the insert performance significantly.

Direct-load INSERT updates the indexes of the table, but CREATE TABLE ... AS SELECT only creates a new table which does not have any indexes

A very efficient way to load data into a table is Direct-Path INSERT. Unlike a conventional INSERT, new rows are not applied to the database buffer cache row by row, but written directly to the data files in new data blocks at the end of the table. Because a Direct-Path INSERT bypasses the space management to reuse free space in the existing blocks, it is much faster to insert multiple rows into a target table. It is highly recommended to use Direct-Path INSERT for loading large data sets into a data warehouse database.

Direct-Path INSERT

Oracle Database inserts data into a table in one of two ways:

1. **During conventional INSERT operations**, the database reuses free space in the table, interleaving newly inserted data with existing data. During such operations, the database also maintains referential integrity constraints.
 2. **During direct-path INSERT operations**, the database appends the inserted data after existing data in the table. Data is written directly into datafiles, bypassing the buffer cache. Free space in the table is not reused, and referential integrity constraints are ignored. Direct-path INSERT can perform significantly better than conventional insert.
- Direct Load Insert only works for INSERT INTO .. SELECT Inserts using the VALUES clause will use conventional insert.
 - Direct Load Insert locks the table in exclusive mode. No other session can insert, update, or delete data, or maintain any indexes.
 - Referential Integrity (Foreign Key) constraints and triggers must be disabled before running Direct Path Insert

the major performance hazard is a BAD SELECT Query.

Most of the time DBA's create an index to speed up the execution, but they have limitations to create an index for all sorts of scenarios. If the number of indexes are more than, it will impact performance during DML operations. In such a scenario, there is a need to find a way to write a query in a format that helps speed up the execution and provide a fast response. @ things we have to take care while writing pl/sql code or while looking for performance issues:

[Avoid overhead on CPU](#)
[Avoid overhead on Memory](#)

To avoid CPU Overhead it is necessary to have efficient SQL statements:

- Check for appropriate indexes on the table
- Statistics are up-to-date (Gather Stats timely)
- Analyze the performance of SQL statements using the explain plan and then rewrite the Query if necessary.
- Implement BULK binding (FORALL and BULK COLLECT), in a situation where a table has a huge amount of data and there is frequent SQL statement execution from a PL/SQL block (i.e. LOOP)
- whenever there is a logical expression or multiple conditions separated by AND or OR, always use the least expensive one
- WITH clause and MATERIALIZE hint WITH clause enables us to write our logic as a block and the data extracted in this block act as a temporary table and is available in the cache. So next time when the query is referenced, oracle doesn't need to execute it rather it directly refers the data extracted earlier. Thus, adds on to performance improvement. Always try to create a block of 'WITH' for the query, if the query result is to reference again and again. Adding MATERIALIZE Hint to the select statement of WITH block gives another advantage and helps fast retrieval of data
- Do not use DISTINCT and UNION unless necessary. UNION All can be preferred if possible
- Do not use outer join unless necessary. Try to make an inner join instead of outer join if possible.
- Avoid fetching unnecessary columns in SELECT clause. It places extra loads on the database.
- Use of wildcard (%) should be avoided at the beginning of predicates. e.g. LIKE '%vis%' causes full table scan.
- Avoid using functions (like UPPER, LOWER) in predicates. Rather check for correct data format and compare with the value if possible. e.g. UPPER(t.name)=UPPER(t1.ename)

This adds the extra overhead of conversion and then comparison, adding to performance degradation.

To avoid Memory Overhead:

Oracle stores information in memory and on disk. Memory access is much faster than disk access, so it is better for data requests to be satisfied by access to memory rather than access to disk. For best performance, store as much data as possible in memory rather than on disk. Oracle's memory requirements depend on your application; therefore, you should tune memory allocation after tuning your application and SQL statements.

It Involves :

- Tuning Buffer cache
 - Tuning SGA/PGA
 - Tuning Redo Log Buffer
-
- Group Related Subprograms into Packages. This is always beneficial to call a packaged subprogram instead of standalone.
 - Use Hints if possible

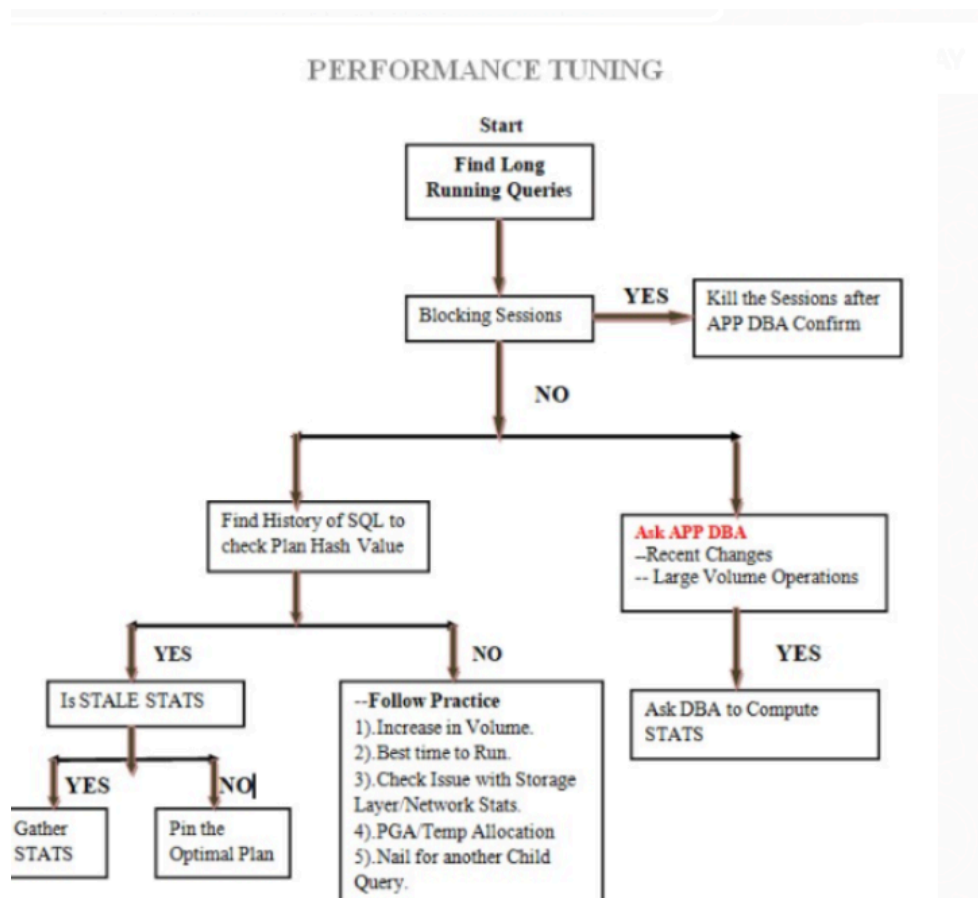
Question: How do I query to find long-running queries?

Answer: You can query the v\$session_longops view to find long-running queries and you can query the AWR to find historical queries. You can view any SQL statement that executes for more than 6 absolute seconds (the "long running" threshold) using the v\$session_longops view.

You can check long running sessions using v\$session_longops script which will show you, % completed, remaining time, so far completed and much more detailed information.

Plan Hash Value: When we execute any sql statement in Oracle, a hash value is being assigned to that sql statement and stored into the library cache. So, that later, if another user requests the same query, then Oracle find the hash value and execute the same execution plan.

STALE Stats : To see if Oracle thinks the statistics on your table are stale, you want to look at the STALE_STATS column in DBA_STATISTICS. If the column returns "YES" Oracle believes that it's time to re-gather stats. However, if the column returns "NO" then Oracle thinks that the statistics are up-to-date.



2 attachments

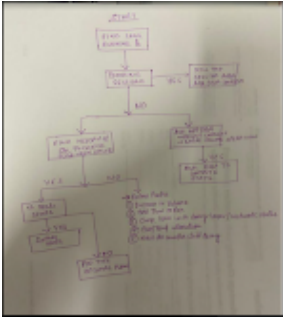


image (9) (3).png
322K

 **FK_Index.sql**
2K